ATELIER C# • COURS PUIS APPLICATION

Créer ton premier menu en C#

Comprendre le fonctionnement d’un menu dans la console, puis le programmer toi-même avec Célestia, Stella et le manoir KonoSekai.

1

AFFICHER

Le menu apparaît.

2

CHOISIR

On lit une réponse.

3

RECOMMENCER

Sauf si on quitte.

Le résultat que tu vas construire

```
===== MENU KONOSEKAI =====
1 - Voir l’energie de Celestia
2 - S’entrainer (+10 energie)
3 - Partir en mission (-20 energie)
0 - Quitter
Ton choix :
```

Ce que tu vas apprendre

- Afficher du texte avec Console.WriteLine.
- Récupérer ce que tape le joueur avec Console.ReadLine.
- Déclencher une action avec switch / case.
- Répéter un menu avec do / while.
- Faire évoluer une valeur avec int et vérifier une condition avec if.

Comment utiliser ce support ?

Lis les pages 2 à 4 en testant les exemples. Fais ensuite les questions des pages 5 et 6 sans regarder de correction. La page 7 sert à vérifier ton programme et ta compréhension. Tu peux me montrer ton code question par question pour qu’on le corrige ensemble.

PARTIE 1 • COURS

Afficher, saisir, mémoriser

Un menu, ce n’est pas une image : c’est du texte affiché, suivi d’une réponse que le programme récupère.

1. Afficher un message

```
Console.WriteLine("Bienvenue au manoir !");
Console.Write("Ton choix : ");
```

WriteLine affiche du texte et passe à la ligne suivante. Write affiche du texte sans retour à la ligne. Les guillemets entourent le texte à afficher. Le point-virgule termine l’instruction.

2. Lire ce que tape le joueur

```
Console.Write("Ton choix : ");
string? choix = Console.ReadLine();
Console.WriteLine("Tu as saisi : " + choix);
```

Console.ReadLine() attend que tu écrives quelque chose, puis que tu appuies sur Entrée. La variable choix conserve la réponse. Le symbole ? indique que la saisie peut aussi être absente.

Point important : "1" et 1 ne sont pas pareils

"1" est du texte (un string). 1 est un nombre entier (un int). Quand tu lis avec ReadLine(), tu récupères du texte. Il est donc tout à fait normal de tester case "1" dans ton menu.

3. Garder une valeur dans une variable

```
int energie = 30;
energie = energie + 10;   // Maintenant : 40
energie += 10;            // Maintenant : 50
Console.WriteLine("Energie : " + energie);
```

Une variable int stocke un nombre entier. += 10 est une écriture raccourcie de energie = energie + 10. Cette valeur peut changer pendant l’exécution du menu.

Mini-vérification

Si le joueur tape 3, quelle valeur prend choix ? Si energie vaut 30 puis que tu exécutes energie += 10, combien vaut-elle ?

PARTIE 2 • COURS

Choisir une action avec switch

Le joueur saisit une option. Le programme doit savoir quel bloc exécuter en fonction de cette réponse.

1. Le rôle de switch / case

```
string? choix = Console.ReadLine();
switch (choix)
{
    case "1":
        Console.WriteLine("Bonjour, Stella !");
        break;

    case "2":
        Console.WriteLine("Bonjour, Akari !");
        break;

    default:
        Console.WriteLine("Choix inconnu.");
        break;
}
```

switch (choix) examine le contenu de choix. case "1" ne s'exécute que si la réponse est "1". break termine ce cas et sort du switch. default gère toutes les autres réponses.

Exemple concret

Si tu écris 2, le programme affiche « Bonjour, Akari ! ». Si tu écris 9, il affiche « Choix inconnu. ». Si tu écris 1, seul le bloc du premier cas est exécuté.

2. Poser une condition avec if

```
int energie = 15;

if (energie >= 20)
{
    Console.WriteLine("Mission autorisee !");
}
else
{
    Console.WriteLine("Energie insuffisante.");
}
```

if exécute son bloc quand la condition est vraie. else s'exécute dans le cas contraire. Ici, >= veut dire « supérieur ou égal à ». On utilisera ce test pour éviter de lancer une mission sans énergie suffisante.

Mini-vérification

Avec une énergie de 15, quel message s'affiche ? Et avec 20 ? Que se passe-t-il si tu entres bonjour dans le switch ?

PARTIE 3 • COURS

Répéter le menu avec une boucle

Sans boucle, un menu ne fonctionne qu'une seule fois. Avec do / while, il revient automatiquement après chaque action.

1. La structure do / while

```
string? choix;
do
{
    Console.WriteLine("1 - Saluer");
    Console.WriteLine("0 - Quitter");
    Console.Write("Ton choix : ");
    choix = Console.ReadLine();

    switch (choix)
    {
        case "1":
            Console.WriteLine("Salut !");
            break;
        case "0":
            Console.WriteLine("A bientot !");
            break;
        default:
            Console.WriteLine("Choix inconnu.");
            break;
    }
}
while (choix != "0");
```

Les instructions entre do { … } sont exécutées au moins une fois. Ensuite while (choix != "0") demande de recommencer tant que le choix est différent de "0". Le symbole != signifie « différent de ».

Attention à la différence

break quitte le switch, pas toute la boucle. C'est la condition du while qui détermine si le menu doit réapparaître. Autrement dit, pour quitter, il faut que choix soit égal à "0".

2. Visualiser l'exécution

- Le menu s'affiche, puis le programme attend la saisie.
- Le switch exécute le cas choisi, puis se termine.
- Si le choix vaut "0", la boucle s'arrête. Sinon, le menu s'affiche de nouveau.

Mini-vérification

À ton avis, si tu tapes successivement 1, 9, puis 0, combien de fois le menu s'affiche-t-il ? Pourquoi ?

PARTIE 4 • TON TP

À toi de créer le menu

Mission : réaliser le menu KonoSekai vu en page 1. Ouvre un projet Console C# et écris ton code dans Program.cs.

Règle du TP

Tu construis le programme étape par étape. Commence par faire fonctionner une question avant d'attaquer la suivante. Le cours te montre les mécanismes ; le TP te demande de les réutiliser dans une situation différente.

QUESTION 1 | Afficher un menu

Affiche le titre « MENU KONOSEKAI » et les quatre choix : 1 Voir l'énergie, 2 S'entraîner, 3 Partir en mission, 0 Quitter. Termine par l'invite Ton choix :.

Repère dans le cours : Page 2 : Console.WriteLine et Console.Write.

Pour valider : Les quatre options s'affichent dans le bon ordre.

QUESTION 2 | Réagir aux choix

Lis la saisie du joueur dans une variable choix. Utilise un switch : option 1 affiche « Affichage de l'énergie », option 2 affiche « Entraînement », option 3 affiche « Mission », option 0 affiche « Au revoir ! ». Toute autre saisie affiche « Choix invalide ».

Repère dans le cours : Page 2 : ReadLine ; page 3 : switch / case / default.

Pour valider : Chaque entrée déclenche uniquement l'action qui correspond.

QUESTION 3 | Faire revenir le menu

Modifie ton programme pour que le joueur puisse effectuer plusieurs actions d'affilée. Le menu doit continuer tant qu'il n'a pas choisi 0.

Repère dans le cours : Page 4 : do / while et !=.

Pour valider : Tester 1, puis 2, puis 0 doit fonctionner sans relancer le programme.

Point de contrôle après la question 3

```
===== MENU KONOSEKAI =====
1 - Voir l'energie
2 - S'entrainer
3 - Partir en mission
0 - Quitter
Ton choix : 2
Entrainement

===== MENU KONOSEKAI =====
...
```

PARTIE 5 • TON TP

Donner une vraie utilité aux choix

Maintenant, les options vont modifier la situation de Célestia. On garde le même menu et on ajoute des règles.

QUESTION 4 | Gérer l'énergie de Célestia

Crée une variable entière energie qui commence à 30, avant la boucle. L'option 1 doit afficher sa valeur actuelle. L'option 2 ajoute 10 points d'énergie, sans jamais dépasser 100. Affiche ensuite la nouvelle valeur.

Repère dans le cours : Page 2 : int et += ; page 3 : if / else.

Pour valider : À 90, un entraînement donne 100. À 100, un entraînement reste à 100.

QUESTION 5 | Lancer une mission

L'option 3 coûte 20 points d'énergie. Si Célestia a au moins 20 points, enlève 20 et affiche « Mission terminée » avec l'énergie restante. Sinon, affiche « Énergie insuffisante » et ne change pas l'énergie.

Repère dans le cours : Page 3 : if (energie >= 20) ; page 2 : modifier une variable.

Pour valider : Avec 30 : mission réussie, énergie 10. Une deuxième mission doit être refusée.

QUESTION 6 | Gérer les mauvaises saisies

Assure-toi que taper 8, bonjour ou une ligne vide affiche « Choix invalide » sans fermer le programme. Le joueur doit pouvoir réessayer immédiatement.

Repère dans le cours : Page 3 : default ; page 4 : la boucle continue tant que le choix n'est pas "0".

Pour valider : Après une erreur, le menu revient normalement.

Petit indice (seulement si tu bloques)

```
// Cette ligne doit etre AVANT la boucle :
int energie = 30;

// Pour limiter l'energie :
if (energie + 10 <= 100)
{
    energie += 10;
}
else
{
    energie = 100;
}
```

Pourquoi avant la boucle ?

Si tu remets int energie = 30; à chaque nouveau tour, Célestia récupère toujours 30 points au lieu de conserver le résultat de ses actions.

PARTIE 6 • VALIDATION

Teste ton programme comme un développeur

Un menu est réussi quand il fonctionne avec un choix normal, plusieurs actions successives et des entrées inattendues.

Scénarios de test à effectuer

Entrées successives	Résultat attendu
1 puis 0	Affiche 30 points, puis quitte.
2 puis 1 puis 0	Entraînement : énergie 40 ; l'option 1 affiche 40.
3 puis 3 puis 0	Première mission : 10 ; deuxième mission refusée.
2 répété 8 fois, puis 1	L'énergie atteint 100 et ne dépasse pas 100.
bonjour puis 1 puis 0	Erreur, retour au menu, énergie affichée, puis sortie.
0	Le programme s'arrête directement après le message de sortie.

Vérifie que tu as compris

- 1. Pourquoi écrit-on case "1" plutôt que case 1 ici ?
- 2. Quelle différence y a-t-il entre break et la condition du while ?
- 3. Pourquoi energie doit-elle être déclarée avant la boucle ?
- 4. Comment empêches-tu l'énergie de descendre sous 0 pendant une mission ?

Défi bonus (facultatif)

Crée une méthode AfficherMenu() qui contient seulement les instructions d'affichage du menu. Appelle-la dans ta boucle à la place de répéter toutes les lignes de Console.WriteLine. Objectif : rendre ton code plus lisible, sans changer son comportement.

Et après ?

Envoie-moi ton Program.cs ainsi que le numéro de la question où tu t'es arrêté. Je vérifierai ton code par rapport à chaque consigne, puis je te poserai une petite question pour m'assurer que le principe est compris avant de passer à la suite.